

Multiprocessadores



UNIFEI

Universidade Federal de Itajubá

IMC – Instituto de Matemática e Computação

Prof. Carlos Minoru Tamaki

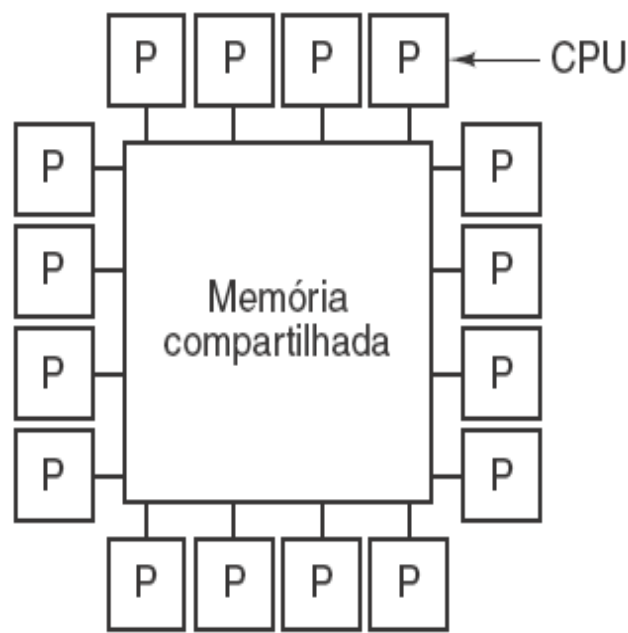
Multiprocessadores de memória compartilhada

- Veremos como CPU's totalmente desenvolvidas podem formar sistemas maiores.
- Sistemas com várias CPU's podem ser divididas em multiprocessadores e multicomputadores.

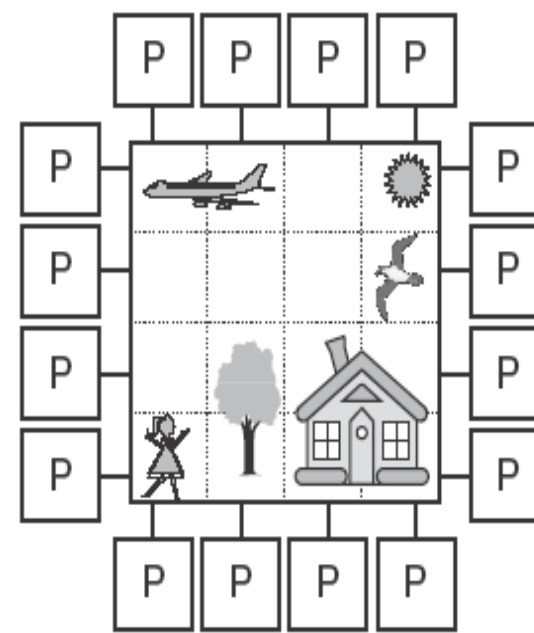
Multiprocessadores

Quando cada CPU tem igual acesso a todos os módulos de memória e a todos os dispositivos de E/S e é tratado pelo sistema operacional como intercambiável com as outras, o sistema é denominado SMP (Symmetric MultiProcessor)

(a) Multiprocessador com 16 CPUs que compartilham uma memória comum. (b) Imagem repartida em 16 seções, cada qual analisada por uma CPU diferente.



(a)

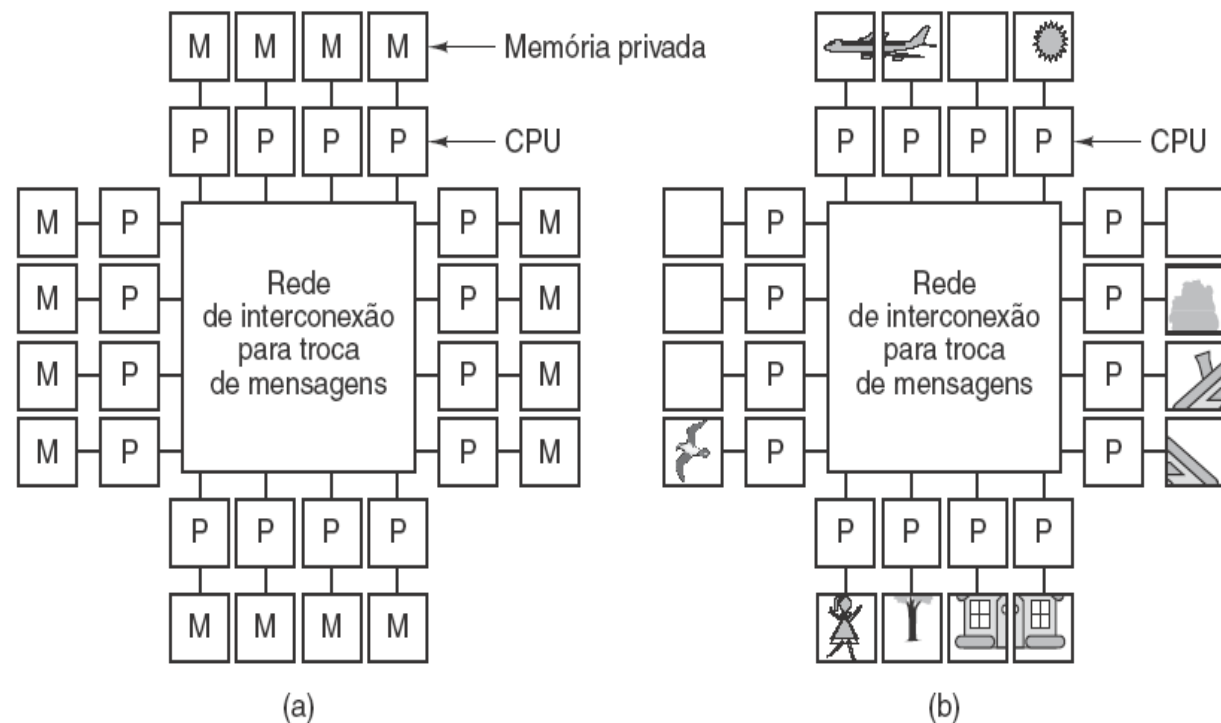


(b)

Multicomputadores

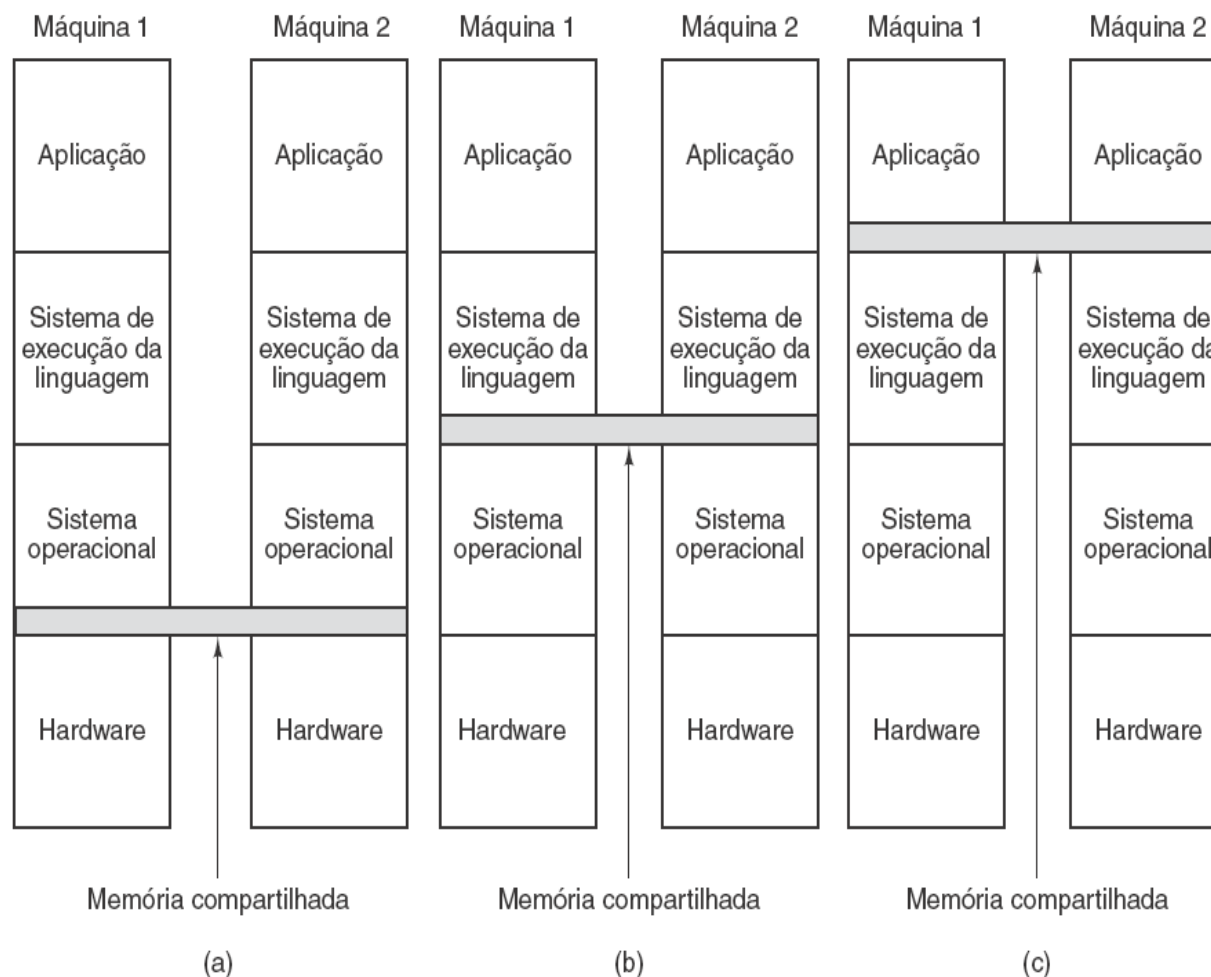
O projeto possível para uma arquitetura paralela é um projeto no qual cada CPU tem sua própria memória privada, acessível somente a ela e a nenhuma outra CPU. Esse projeto denominado multicomputador ou sistema de memória distribuída.

- (a) Multicomputador com 16 CPUs, cada uma com sua própria memória privada.
(b) Imagem de mapa de bits da Figura 8.15 subdividida entre as 16 memórias.



Outra possibilidade é usar o hardware de multicomputador e fazer com que o sistema operacional simule memória compartilhada, único espaço de endereço virtual de compartilhamento de páginas, **DSM (distributed Shared Memory – Memória compartilhada distribuída)**

Várias camadas onde a memória compartilhada pode ser implementada.
 (a) Hardware.
 (b) Sistema operacional.
 (c) Sistema de execução da linguagem.

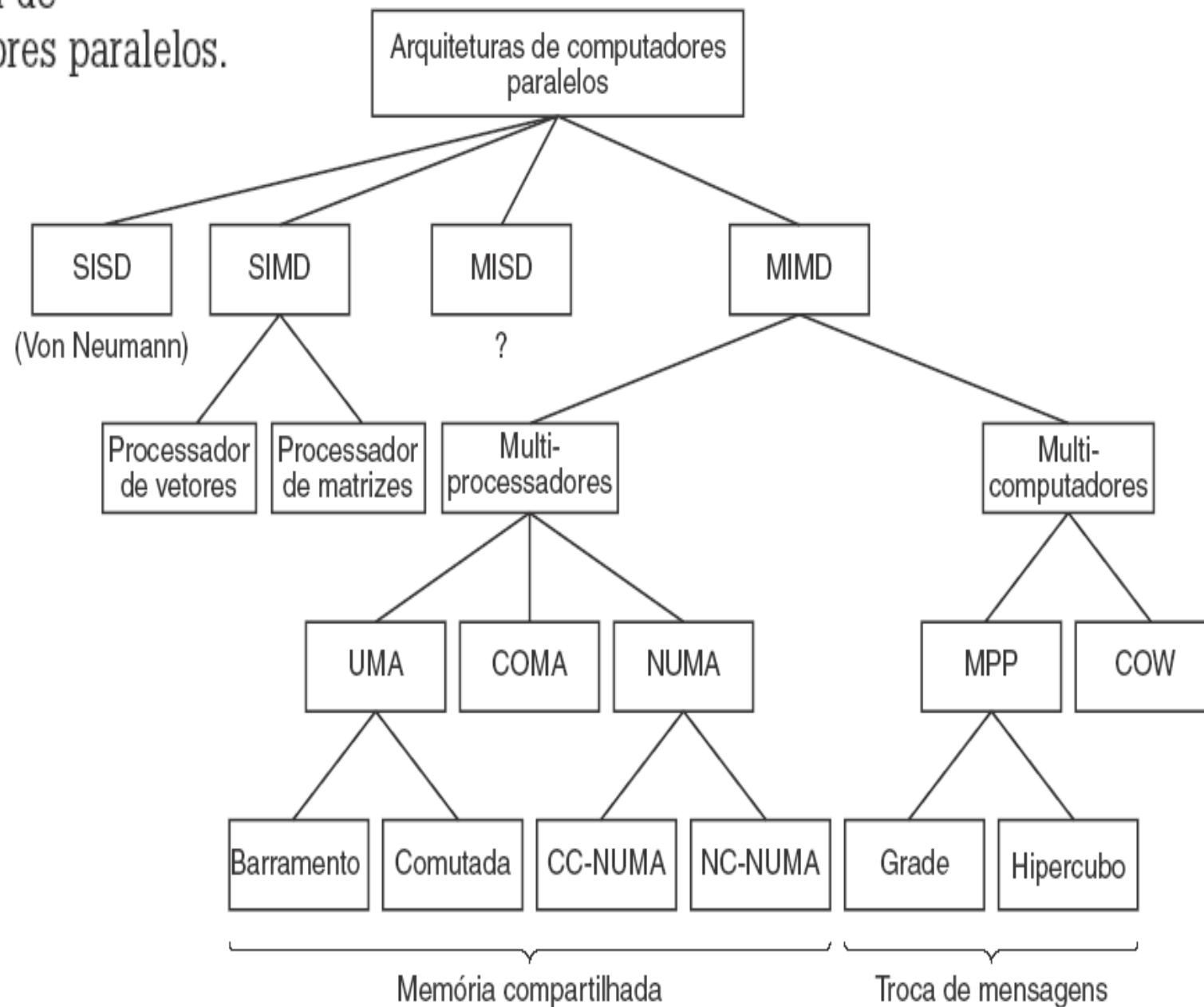


Taxonomia de computadores paralelos

Taxonomia de Flynn computadores paralelos

Fluxos de instruções	Fluxos de dados	Nome	Exemplos
1	1	SISD	Máquina clássica de Von Neumann
1	Múltiplos	SIMD	Supercomputador vetorial, processador de <i>array</i>
Múltiplos	1	MISD	Possivelmente nenhum
Múltiplos	Múltiplos	MIMD	Multiprocessador, multicomputador

Taxonomia de computadores paralelos.



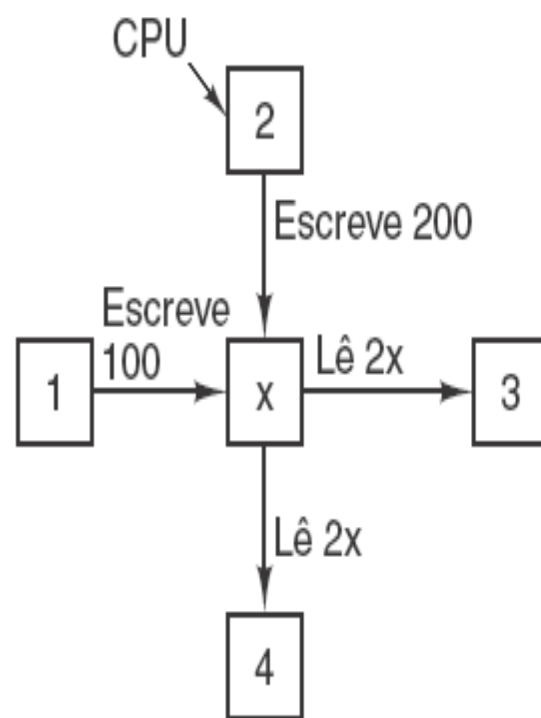
Semântica de Memória

- Um modo de ver a semântica de memória é como um contrato entre software e hardware de memória (Adve e Hill, 1990). Se o software concordar em obedecer a certas regras, a memória concorda em entregar certos resultados e a discussão fica centrada em quais são as regras. Elas são denominadas **modelos de consistência**.
 - **Consistência Estrita**: qualquer leitura para uma localização x sempre retorna o valor da escrita mais recente para x. Este modelo é efetivamente impossível de implementar de qualquer outro modo que não seja ter um único módulo de memória que atende a todas as requisições segundo a política: primeiro que chega é o primeiro a ser atendido, sem cache nem duplicação de dados.
 - **Consistência Sequencial**: na presença de múltiplas requisições de leitura e escrita, o hardware escolhe alguma intercalação de todas as requisições, mas todas as CPU's vêem a mesma ordem.

Consistência sequencial

(a) Duas CPUs escrevendo e duas CPUs lendo uma palavra de memória em comum.

(b)–(d) Três modos possíveis de intercalar as duas escritas e as quatro leituras em relação ao tempo.



(a)

W100

W200

R3 = 200

R3 = 200

R4 = 200

R4 = 200

(b)

W100

R3 = 100

W200

R4 = 200

R3 = 200

R4 = 200

(c)

W200

R4 = 200

W100

R3 = 100

R4 = 100

R3 = 100

(d)

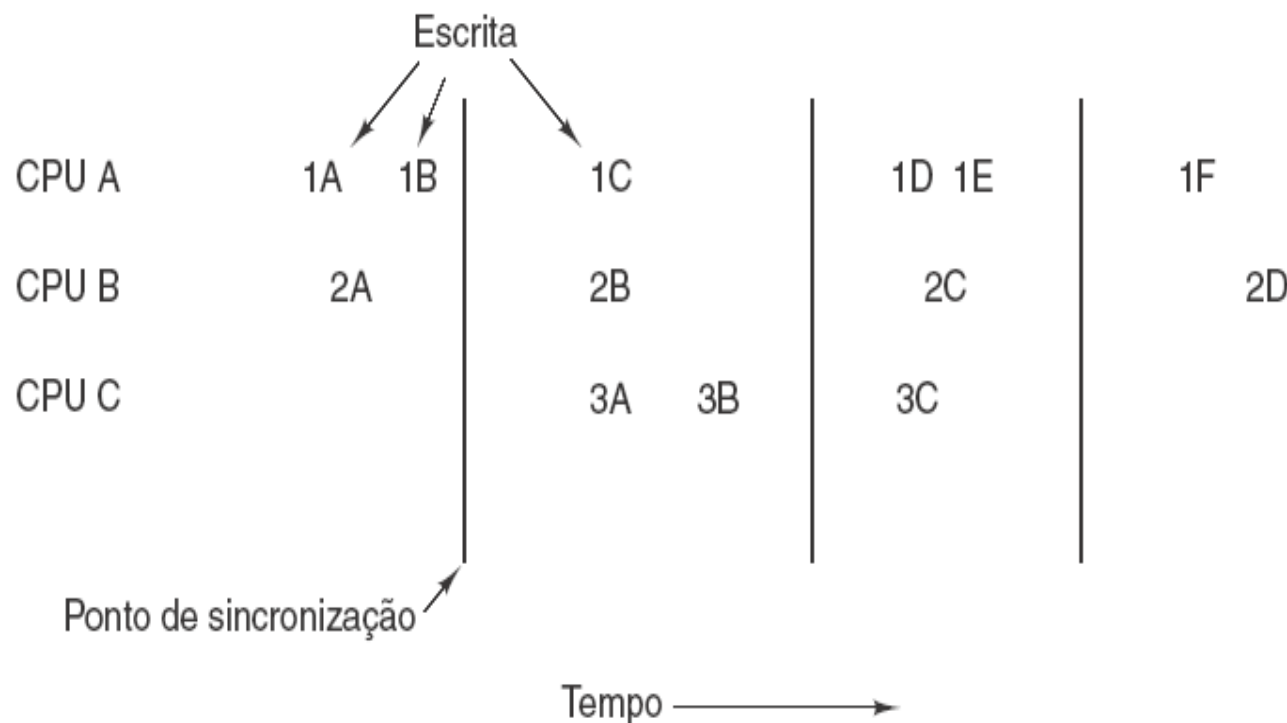
Consistência de processador

- Propriedades:
 1. Escritas por qualquer CPU são vistas por todas as CPU's na ordem em que foram emitidas.
 2. Para cada palavra de memória, todas as CPU's vêem todas as escritas para ela na mesma ordem.

Consistência fraca

A memória fracamente consistente usa operações de sincronização para dividir o tempo em épocas sequenciais.

Nem mesmo garante que escritas de uma única CPU sejam vistas em ordem.



Consistência de liberação

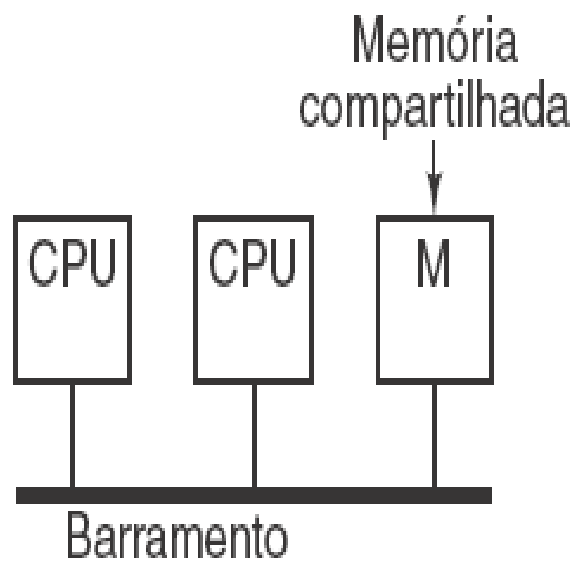
- A consistência fraca tem problema de ser ineficiente porque deve encerrar todas as operações de memória pendentes e deter todas as novas até que as operações correntes tenham terminado. A consistência de liberação melhora as coisas adotando um modelo semelhante ao das seções críticas (Charachorloo et al., 1990).
- A idéia que fundamenta esse modelo é que, quando um processo sai de uma região crítica não é necessário forçar todas as escritas a concluírem imediatamente. Basta assegurar que elas estejam encerradas antes que qualquer processo entre naquela região crítica novamente.

Arquiteturas simétricas de multiprocessador UMA

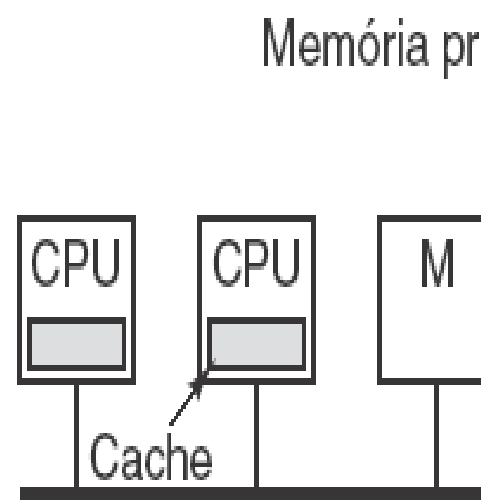
Três multiprocessadores baseados em barramento.

(a) Sem cache. (b) Com cache. (c) Com cache e memórias privadas.

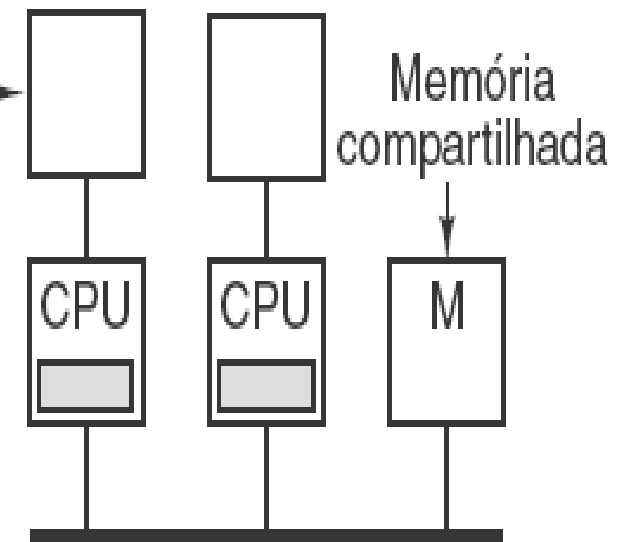
UMA – Uniform
Memory Access



(a)



(b)



(c)

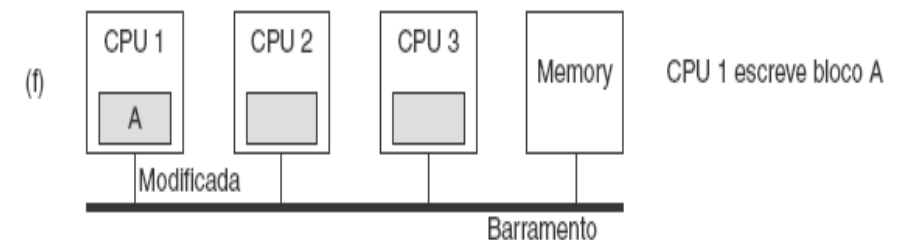
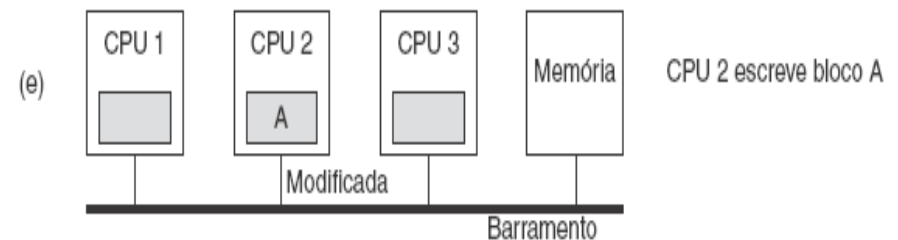
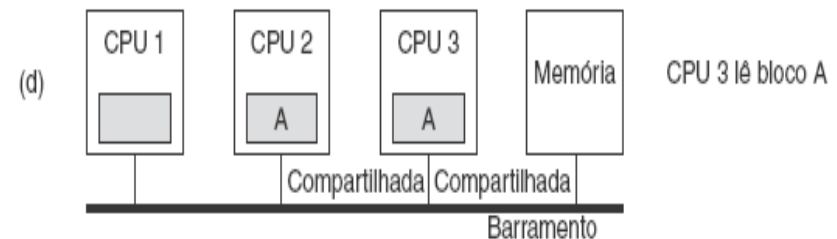
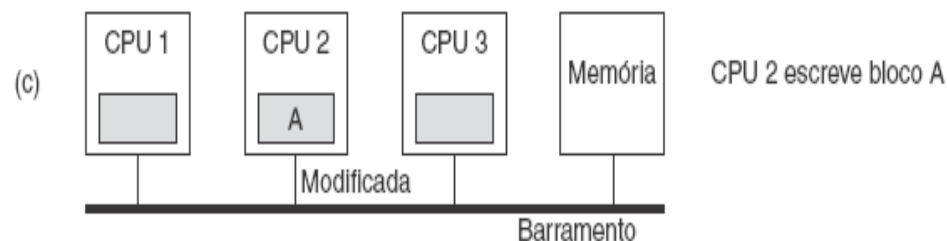
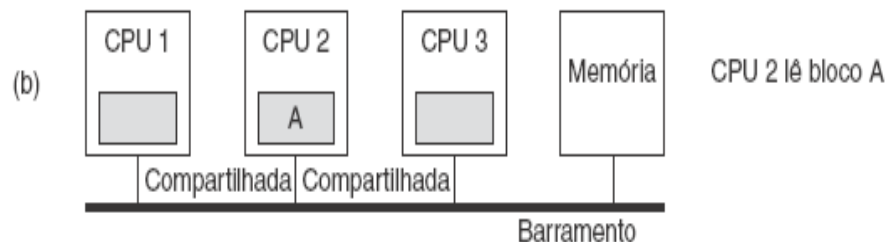
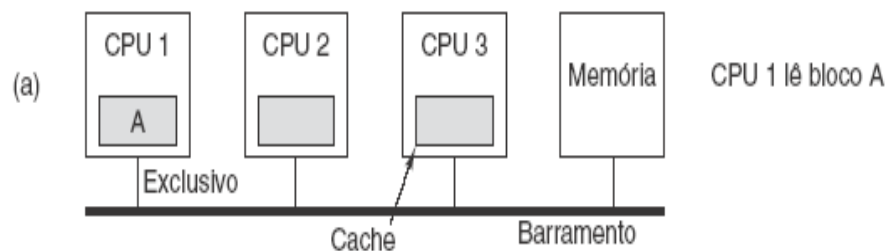
Caches de escuta

Protocolo de coerência de cache de escrita direta. Os retângulos vazios indicam que nenhuma ação foi realizada.

Ação	Requisição local	Requisição remota
Ausência da <i>cache</i> para leitura	Busque dados da memória	
Presença na <i>cache</i> para leitura	Use dados da <i>cache</i> local	
Ausência da <i>cache</i> para escrita	Atualize dados na memória	
Presença na <i>cache</i> para escrita	Atualize <i>cache</i> e memória	Invalide entrada de <i>cache</i>

O protocolo MESI de coerência de cache

- **Inválido** – A entrada da cache não contém dados válidos.
- **Compartilhado (shared)** – múltiplas caches podem conter a linha; a memória está atualizada.
- **Exclusivo** – nenhuma outra cache contém a linha; a memória está atualizada.
- **Modificado** – a entrada é válida; a memória é inválida; não existem cópias.



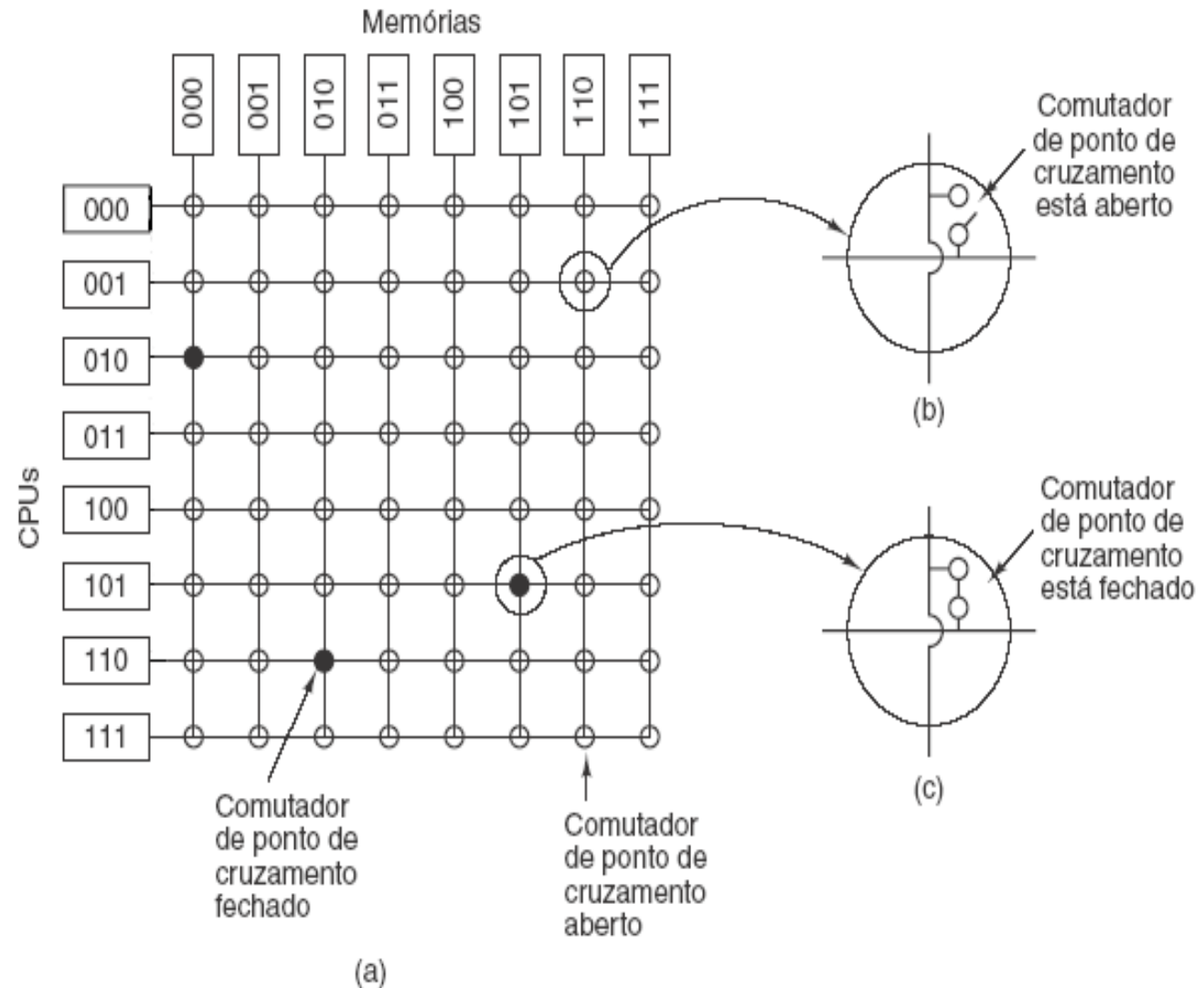
Multiprocessadores UMA usando comutadores Crossbar

(a) Comutador crossbar

8×8 .

(b) Ponto de cruzamento aberto.

(c) Ponto de cruzamento fechado.



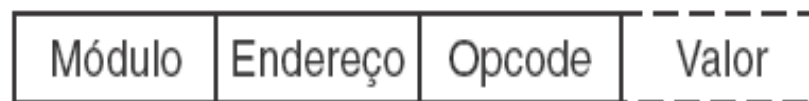
Multiprocessadores UMA usando redes de comutação multiestágios

(a) Comutador 2×2 .

(b) Formato de mensagem.



(a)



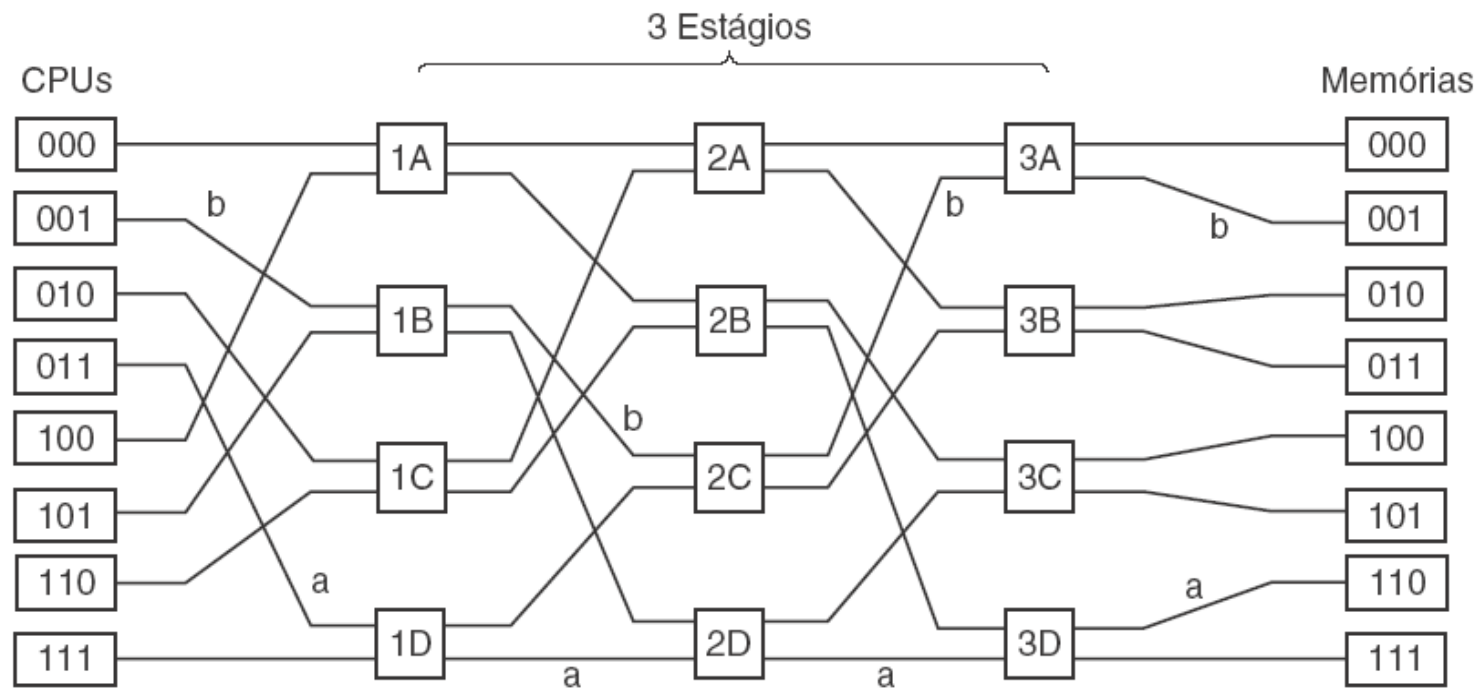
(b)

Este comutador opera seguindo a seguinte regra, se o bit correspondente ao comutador for:

- 0 – utiliza a saída x
- 1 – utiliza a saída y

Rede de computação Ômega

- a) O processador 011 deseja acessar a memória 110
- b) O processador 001 deseja acessar a memória 001

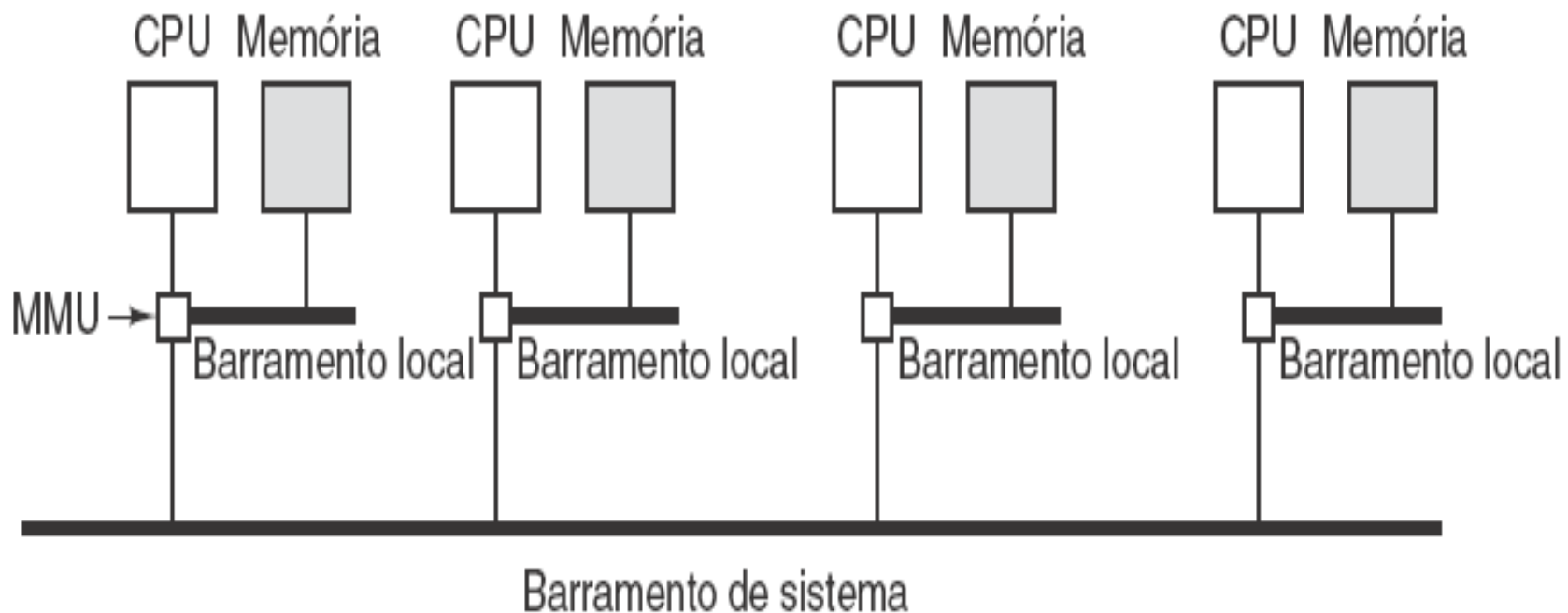


Multiprocessadores NUMA

Máquina NUMA com dois níveis de barramentos o Cm* foi o primeiro multiprocessador a usar esse projeto

NUMA – Non-Uniform Memory Access

- 1) Há um único espaço de endereço visível a todas as CPUs.
- 2) O acesso à memória remota é feito usando instruções LOAD e STORE.
- 3) O acesso à memória remota é mais lento do que o acesso à memória local.

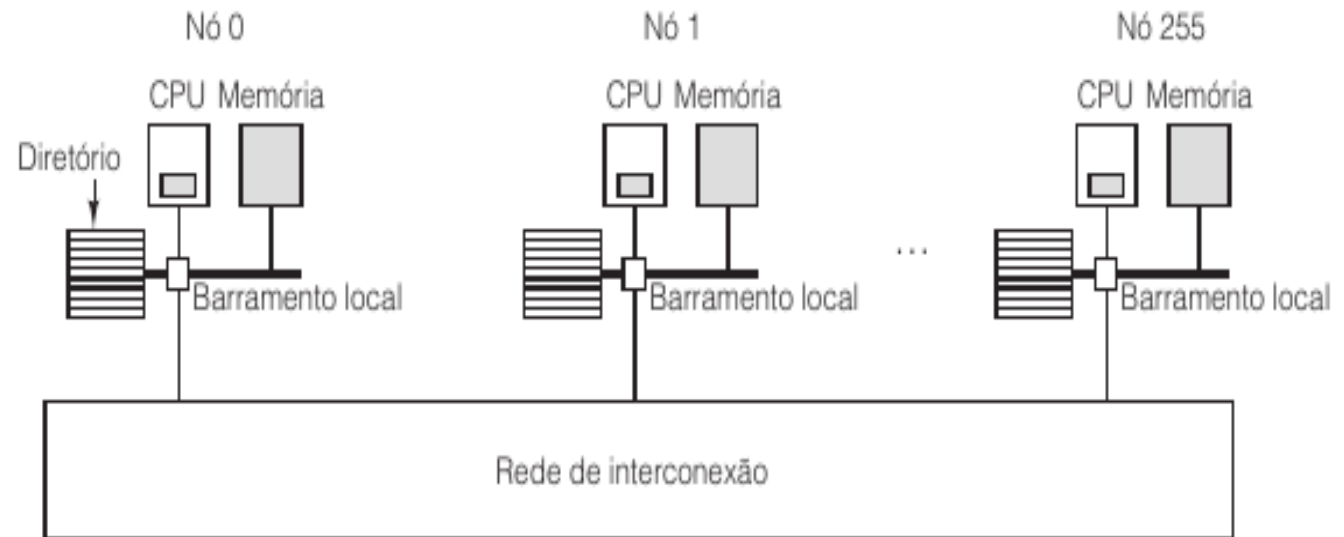


Multiprocessadores NUMA com coerência de cache

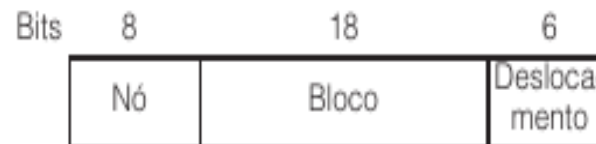
(a) Multiprocessador de 256 nós baseado em diretório.

(b) Divisão de um endereço de memória de 32 bits em campos.

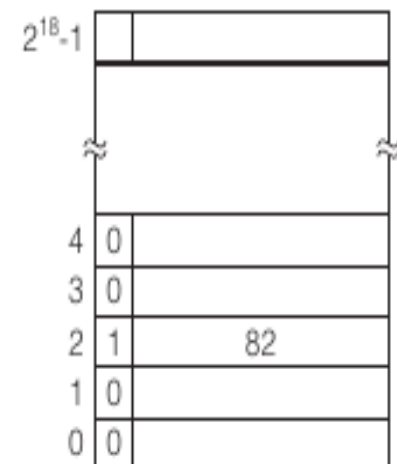
(c) Diretório no nó 36.



(a)



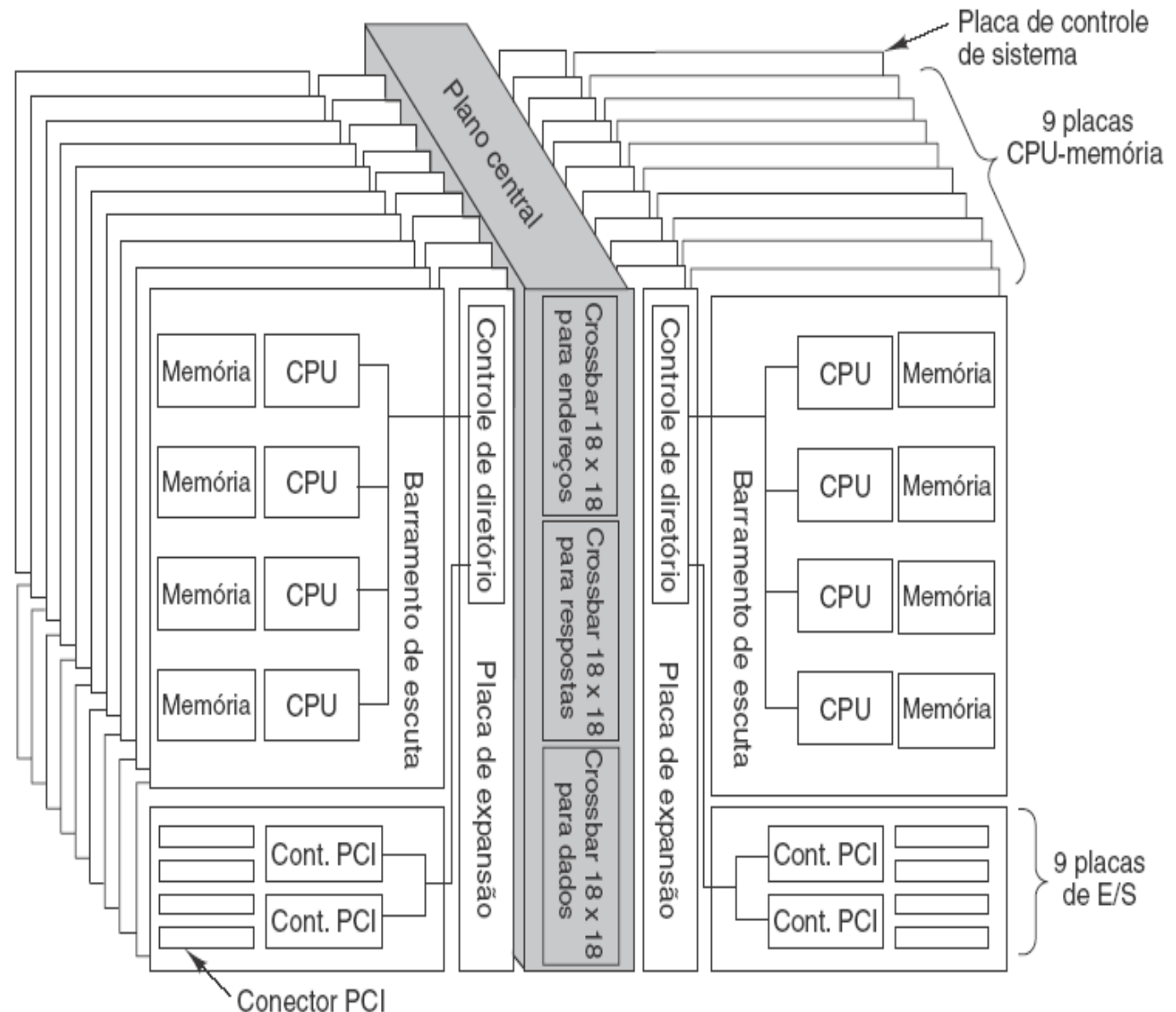
(b)



(c)

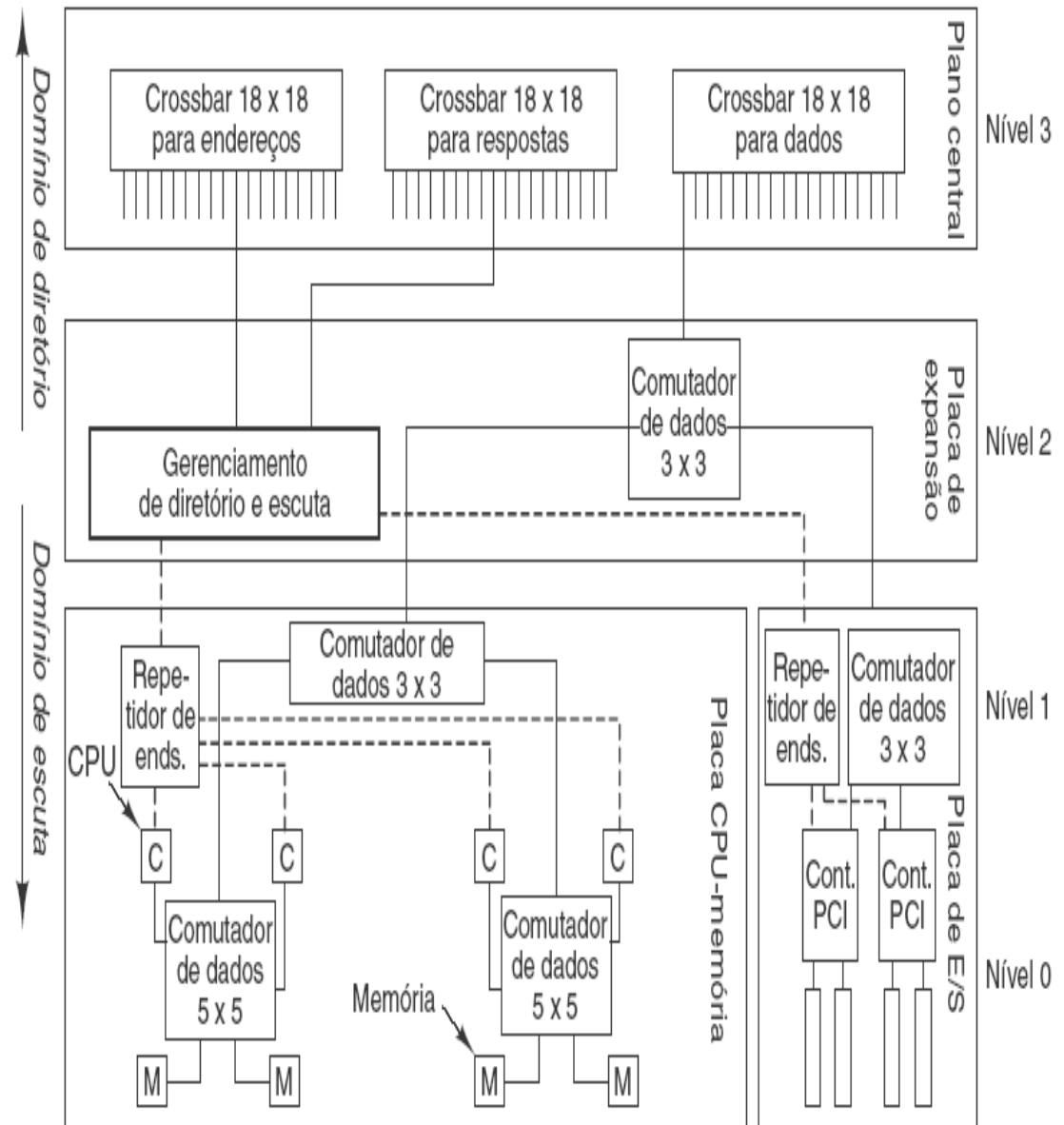
O Multiprocessor NUMA Sun Fire E25K

Multiprocessador Sun
Microsystems E25K.



O Sun Fire E25K usa uma interconexão de quatro níveis. As linhas tracejadas são caminhos de endereços. As linhas cheias são caminhos de dados. Cada bloco de memória e linha de cache de cada chip de CPU pode estar em um de três estados:

- 1) Acesso exclusivo (para escrita).
- 2) Acesso compartilhado (para leitura).
- 3) Inválido (isto é, vazio).



Multiprocessadores COMA

- COMA – Cache Only Memory Access
- As desvantagens das máquinas NUMA e CC-NUMA é que referência a memória remota é muito mais lenta que referências à memória local.
- O projeto COMA tenta resolver todos os problemas de acesso à memória remota utilizando a memória principal como cache.
- O espaço de endereço físico é subdividido em linhas de cache, que migram pelo sistema sob demanda.
- Uma memória que apenas atrai linhas conforme necessário é denominado memória de atração.

- Usar a memória principal como cache aumenta muito a taxa de presença no cache.
- Sistemas COMA introduzem dois novos problemas:
 - Como as linhas de cache são localizadas?
 - Quando uma linha é expurgada da memória, o que acontece se ela for a última cópia?
- Esses problemas são resolvidos somente com a adição de um hardware especial.
- Uma outra solução é mapear as páginas inteiras mas não exige que todas as linhas de cache estejam presentes
- Nesta solução, o hardware precisa de um mapa de bits por página que dê um único bit por linha de cache indicando a presença ou ausência da linha, Este projeto foi denominado COMA Simples.

Bibliografia

- Organização Estruturada de Computadores
– Andrew S. Tanenbaum, Tood Austin – 6ª
Edição 2013 Prentice Hall
- Vários sites da internet